

Lab 4 Report

Machine Learning for Hardware Design

Authors: Zach Dixon, Jongwoo Kim

Team Number: 06

Part 3. Quantized Neural Network Software Implementation

1.1) Estimate the memory size needed to store the TinyImageNet parameters

Layers	Weight	bias
conv1	$5*5*3*32 \rightarrow 2,400$	32
conv2	$5*5*32*32 \rightarrow 25,600$	32
conv3	$3*3*32*64 \rightarrow 18,432$	64
conv4	$3*3*64*64 \rightarrow 36,864$	64
conv5	$3*3*64*64 \rightarrow 36,864$	64
conv6	$3*3*64*128 \rightarrow 73,728$	128
dense1	$2048*256 \rightarrow 524,288$	256
dense2	$256*200 \rightarrow 51,200$	200
Total	769,376	840

Table 1: Number of weights and biases per layer.

The model has 769,376 weights and 840 biases (770,216 parameters). Assuming int32 biases and 1 KB = 1000 B:

- fp32: $770,216 \times 4 \text{ B} \approx 3,081 \text{ KB}$

- int8 weights: $769,376 \times 1 \text{ B} + 840 \times 4 \text{ B} \approx 773 \text{ KB}$

- int2 weights: $769,376 \times 0.25 \text{ B} + 840 \times 4 \text{ B} \approx 196 \text{ KB}$

1.2) Which is the max quantization level among 1,2,4,8,16 bits that would allow the TinyImageNet model to fit on a device with 1000 KB of overall memory?

Max quantization level for 1000 KB: 16-bit weights need 1,542 KB, which does not fit. 8-bit weights need 773 KB, which fits. The maximum level is 8 bits.

(4-bit: 388 KB, 2-bit: 196 KB, and 1-bit: 99 KB also fit, but 8 bits is the highest.)

1.3) Are there any other types of memory than the raw model memory that needed to be taken into account to enable inference?

Yes. Inference also needs activation buffers (the input and output feature maps of each layer), the input image, per-layer scale/zero-point values, and program code/stack. Activations are the largest of these. Our framework allocates all layer outputs at once, which takes 362 KB in int8, so the 8-bit model plus activations totals about 1,134 KB and does not fit. (This is an estimate; Sections 6.1 and 6.3 give the exact numbers for our implementation.)

1.4) What impact could quantization have on this memory size (please quantify for your model)?

Memory scales with bit-width. fp32 $\rightarrow$ int8 is about 4 $\times$ smaller (3,081 $\rightarrow$ 773 KB), and fp32 $\rightarrow$ int2 is about 16 $\times$ smaller ($\rightarrow$ 196 KB). Activations also shrink 4 $\times$ in int8 (1,447 $\rightarrow$ 362 KB).

2.1) Plot the histogram

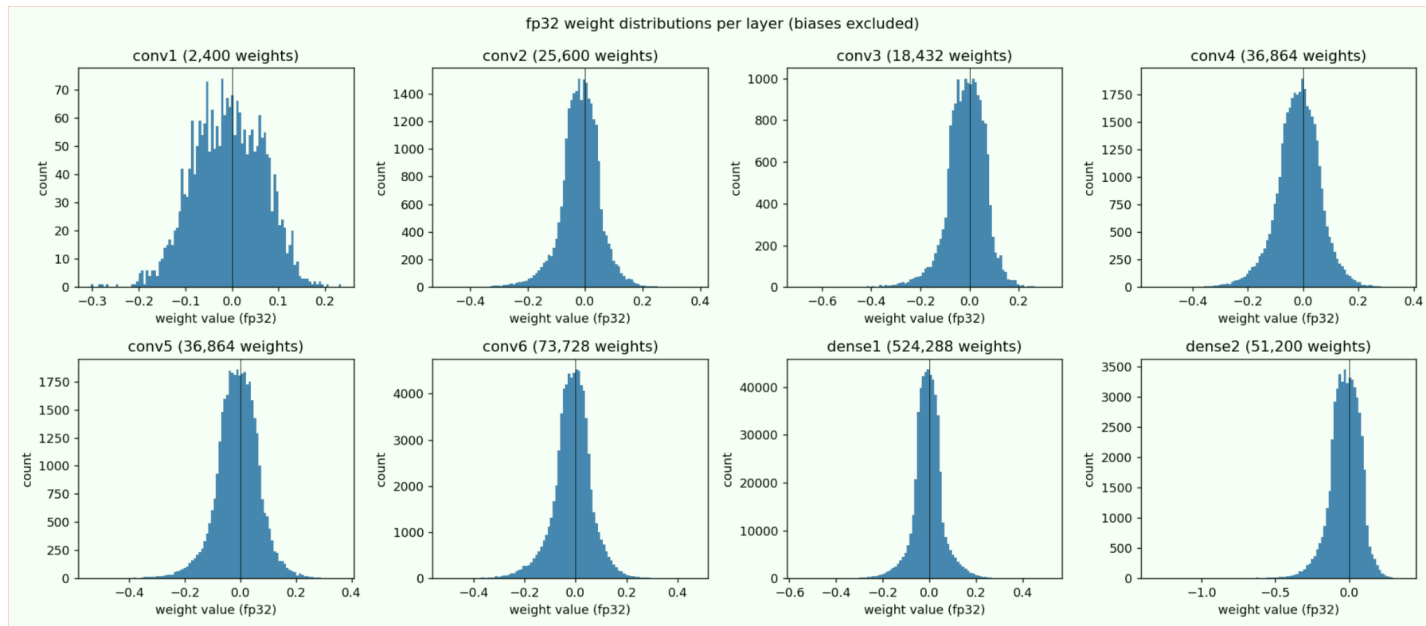


Figure 1: fp32 weight histograms for all layers with weights (conv1–6, dense1–2). MaxPool/Flatten have no weights; biases are not plotted.

2.2) What do you observe from the distribution?

Every layer has a bell-shaped distribution centered near zero (mean ≈ -0.01 , std ≈ 0.06 – 0.11), with long, sparse tails. About 98–99% of weights lie within $\pm 3\sigma$, while the extreme values reach 4– 12σ (for example, dense2 min = -1.32 with $\sigma = 0.11$). The distributions are also slightly skewed toward negative values. conv1 is the exception: it has few weights and a wider, flatter shape.

2.3) Are the weight values uniformly distributed?

No. Most weights are concentrated near zero, and only a few outliers set the range. Because the scale is set by $\max|W|$, these rare outliers stretch the range: in dense1, 74% of the weights fall within 10% of $\max|W|$, and in dense2 84% do. Those weights end up on only a few integer levels near zero, so precision is lost even at 8 bits, and the loss grows at 4 and 2 bits. Weight and bias ranges also differ between layers, which is why per-layer scales are needed.

2.4) Show & note the min-max values of both weights and biases

Layer	W min	W max	B min	B max
conv1	-0.3029	0.2343	-0.0573	0.0976
conv2	-0.4868	0.3826	-0.0702	0.0956
conv3	-0.6924	0.3248	-0.2099	0.1523
conv4	-0.5415	0.3629	-0.2958	0.1022
conv5	-0.5367	0.3624	-0.3180	0.1807
conv6	-0.5107	0.4682	-0.1361	0.2128
dense1	-0.5576	0.5128	-0.1793	0.1822
dense2	-1.3241	0.3666	-0.2397	0.1636
Table 2: Min and max values of the fp32 weights and biases for each layer.				

3.1) Using a random 1,000 images out of the 100,000 train images, please find the overall min-max value of activations for each layer across the 1,000 input images

Layer	Min	Max	Avg
input	0.0000	1.0000	0.4380
conv2d	0.0000	1.7098	0.0332
conv2d_1	0.0000	3.9521	0.0312
max_pooling2d	0.0000	3.9521	0.0518
conv2d_2	0.0000	2.9215	0.0333
conv2d_3	0.0000	4.1077	0.0415
max_pooling2d_1	0.0000	4.1077	0.0664
conv2d_4	0.0000	5.7699	0.1293
conv2d_5	0.0000	8.9120	0.0916
max_pooling2d_2	0.0000	8.9120	0.1598
flatten	0.0000	8.9120	0.1598

dense	0.0000	21.4512	0.5693
dense_1(softmax)	0.0000	0.9999	0.0050
Table 3: Overall min, max, and average activation of each layer over 1,000 random training images (inputs normalized to [0, 1]).			

We sampled 1,000 random training images (seed 487), normalized them to [0, 1], and recorded each layer's overall min, max, and average activation. All minimums are 0 because of ReLU, and the softmax output stays below 1. The maximum grows with depth (1.71 at conv1 $\rightarrow$ 8.91 at conv6 $\rightarrow$

21.45 at dense1), while the averages stay small (≤ 0.57). So most activations are near zero with a few large outliers, and each layer needs its own scale. The averages will be used for the zero-points in Section 4.

Part 4. Implementing Basic Linear Quantization

4.1) Use all of the above provided information to implement 8 bit quantized inference within your C++ framework

We implemented 8-bit post-training quantization in our C++ framework. In Python, we computed each layer's input scale and zero-point from the Section 3.3 activation ranges and its weight scale from $\max|W|$. We then exported int8 weights and int32 biases ($S_b = S_i \cdot S_w$). Each conv/dense layer accumulates $(\text{input} - z_i) \cdot \text{weight}$ in int32, dequantizes the result, applies ReLU, and requantizes it to int8 for the next layer. MaxPool and Flatten work directly on int8, and the last layer outputs fp32 values for the unchanged softmax. A QUANT_BITS flag switches between the fp32 and quantized builds.

Layer	S_i	z_i	S_w
conv1	225.98	-99	419.31
conv2	75.75	-3	260.90
conv3	32.56	-2	183.43
conv4	43.97	-1	234.51
conv5	31.43	-2	236.64
conv6	22.52	-3	248.70
dense1	14.51	-2	227.77
dense2	6.08	-3	95.91
Table 4: Per-layer quantization parameters			

Layer	Max Error	Cosine similarity
conv1	0.020	0.996
conv2	0.043	0.999
conv3	0.049	0.999
conv4	0.066	0.991
conv5	0.103	0.990
conv6	0.138	0.987
dense1	0.285	0.995
softmax	0.042	0.889
Table 5: 8-bit vs fp32 per-layer error (max error, cosine similarity)		

```

[Info]: --- Quantized vs fp32 reference (image_0, per layer) ---
Opened binary file data/image_0.bin
Opened binary file data/image_0_data/layer_0_output.bin
Layer 0 Convolutional: max error 0.0203158, cosine similarity 0.995731
  DE Quantized value: 0.105614
  Original value: 0.101601
  DE Quantized value: 0.237631
  Original value: 0.235907
  DE Quantized value: 0.250832
  Original value: 0.244445
  DE Quantized value: 0
  Original value: 0.000566754
  DE Quantized value: 0.092412
  Original value: 0.0805964
Opened binary file data/image_0_data/layer_1_output.bin
Layer 1 Convolutional: max error 0.0430197, cosine similarity 0.999351
Opened binary file data/image_0_data/layer_2_output.bin
Layer 2 MaxPooling: max error 0.0430197, cosine similarity 0.999156
Opened binary file data/image_0_data/layer_3_output.bin
Layer 3 Convolutional: max error 0.0486395, cosine similarity 0.998727
Opened binary file data/image_0_data/layer_4_output.bin
Layer 4 Convolutional: max error 0.0663402, cosine similarity 0.990847
Opened binary file data/image_0_data/layer_5_output.bin
Layer 5 MaxPooling: max error 0.0663402, cosine similarity 0.992977
Opened binary file data/image_0_data/layer_6_output.bin
Layer 6 Convolutional: max error 0.102608, cosine similarity 0.989816
Opened binary file data/image_0_data/layer_7_output.bin
Layer 7 Convolutional: max error 0.13783, cosine similarity 0.98748
Opened binary file data/image_0_data/layer_8_output.bin
Layer 8 MaxPooling: max error 0.13783, cosine similarity 0.988538
Opened binary file data/image_0_data/layer_9_output.bin
Layer 9 Flatten: max error 0.13783, cosine similarity 0.988538
Opened binary file data/image_0_data/layer_10_output.bin
Layer 10 Dense: max error 0.285298, cosine similarity 0.994821
Opened binary file data/image_0_data/layer_11_output.bin
Layer 12 SoftMax: max error 0.0419948, cosine similarity 0.888773

```

Figure 2: 8-bit dequantized vs fp32 reference outputs per layer (image_0), including sample "DE Quantized / Original" values.

```

[Info]: --- Top-1 class: quantized vs fp32 reference ---
Opened binary file data/image_0.bin
Opened binary file data/image_0_data/layer_11_output.bin
image_0: quantized top-1 = 163 (p=0.188184), fp32 top-1 = 163 (p=0.14619) MATCH
Opened binary file data/image_1.bin
Opened binary file data/image_1_data/layer_11_output.bin
image_1: quantized top-1 = 160 (p=0.0678728), fp32 top-1 = 160 (p=0.06861) MATCH
Opened binary file data/image_2.bin
Opened binary file data/image_2_data/layer_11_output.bin
image_2: quantized top-1 = 40 (p=0.150726), fp32 top-1 = 40 (p=0.143823) MATCH

```

Figure 3: Top-1 predictions of the 8-bit model match fp32 for all three test images.

Table 5 and Figure 2 show that the 8-bit outputs closely track the fp32 reference. The error grows slowly with depth, but every intermediate layer keeps a cosine similarity of at least 0.987. The softmax output is lower (0.889) because softmax amplifies small logit differences, yet the prediction is unchanged: Figure 3 shows that all three test images give the same top-1 class as fp32 (163, 160, 40). On the lab machine, the 8-bit model also runs about 1.6× faster than fp32 (87.6 ms vs 142.9 ms, averaged over 1,000 validation images).

Part 5. To Quantize Even Further Beyond!

5.1) Extract 1000 validation images and labels

We exported the first 1,000 images of the TinyImageNet validation set (covering all 200 classes) as raw uint8 pixels with their int32 labels. The C++ framework normalizes them by /255, as in Lab 1, and reports top-1/top-10 accuracy (./build/ml val). Our C++ fp32 build reproduces the Keras accuracy exactly (23.5% top-1, 60.5% top-10), which validates the baseline used for the quantized models.

5.2) Implement 2 and 4 bit quantization

For b-bit quantization, the constant 127 in S_i and S_w becomes $2^{(b-1)} - 1$ (7 for 4 bit, 1 for 2 bit), and values are clamped to $[-2^{(b-1)}, 2^{(b-1)} - 1]$. Everything else stays the same, so a single QUANT_BITS preprocessor flag selects 32, 8, 4, or 2 bits.

5.3) Across all of your implementations, 32 bit, 8 bit, 4 bit, and 2 bit, include in your report one or more visualizations of an output channel from a layer of your choice (the same channel, layer, and image across levels), to visualize the effects of quantization. Briefly describe what differences you see across quantization levels

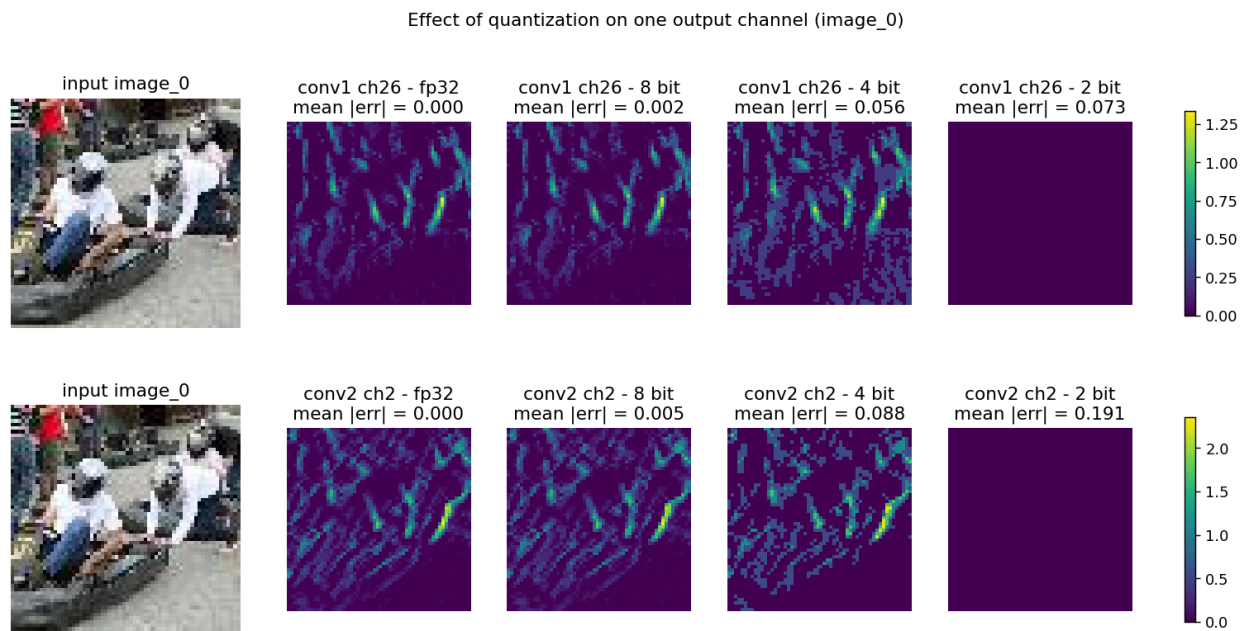


Figure 4: The same output channel (conv1 ch26, conv2 ch2) of image_0 at fp32, 8, 4, and 2 bits, shown with the same color scale.

The 8-bit maps look the same as fp32 (mean error ≤ 0.005). The 4-bit maps keep the main edge structure, but values snap to a few coarse levels and a low background "noise floor" appears where fp32 is zero (mean error 0.06–0.09). At 2 bits the activations can take only four levels, and small activations round to zero, so both channels become completely blank and the signal is lost

from conv2 onward. This matches the final outputs: 8-bit keeps the fp32 prediction, 4-bit is clearly degraded but still structured, and 2-bit collapses to a uniform softmax output ($p = 1/200$ for every class).

5.4) Validation for 2 and 4 bit

As the manual warns, cosine similarity breaks down at low precision (2-bit layers drop to 0), so we validated the 4/2-bit models with mean absolute error, feature-map visualization (Figure 4), and top-1/top-10 accuracy over the 1,000 validation images. 4-bit (6.6% top-1, 24.1% top-10) stays well above random guessing (0.5% / 5%), while 2-bit (0.3% / 4.6%) is at chance level.

Part 6. A Thorough Evaluation

6.1) Report these as a 4x4 table, as well as independently with explanations by the following directions

Model	Storage (on disk)	Memory (minimum / as implemented)	Latency (avg / image)	Accuracy (top-1 / top-10)
fp32	3,081 KB	3,943 KB / 4,527 KB	142.9 ms	23.5% / 60.5%
8 bit	773 KB	988 KB / 1,185 KB	87.6 ms	23.8% / 60.4%
4 bit	773 KB (388 KB if packed)	988 KB / 1,185 KB	87.6 ms	6.6% / 24.1%
2 bit	773 KB (196 KB if packed)	988 KB / 1,185 KB	87.8 ms	0.3% / 4.6%

Table 6: Comparison of the four implementations

Each column is explained in Sections 6.2 (storage), 6.3 (memory), 6.4 (latency), and 6.5 (accuracy). Latency and accuracy were measured on the lab machine over the same 1,000 validation images. The measured peak RSS (/usr/bin/time -v) was 8,096 KiB for fp32 and 4,736 KiB for 8 bit, which includes about 3.7 MB of program/runtime overhead; the fp32–8-bit difference (3.44 MB measured vs 3.34 MB computed) confirms our as-implemented footprint.

6.2) Report how much space each model consumes on-disk. State the formula/method you used to come up with this number for each model, and then compare it with what the filesystem on your computer reports (by highlighting the files in the file explorer, right clicking, and selecting "Properties")

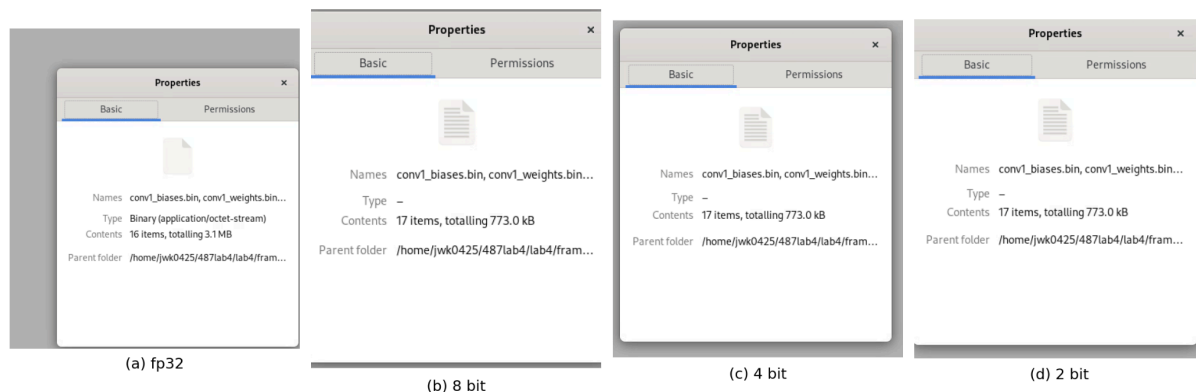


Figure 5: File manager "Properties" of the parameter files loaded by each model: (a) fp32, (b) 8 bit, (c) 4 bit, (d) 2 bit.

Model	Formula	Size	File manager
fp32	$769,376 \times 4 + 840 \times 4$	3,080,864 B	3.1 MB
8 bit	$769,376 \times 1 + 840 \times 4 + 246$	772,982 B	773.0 kB
4 bit	$769,376 \times 1 + 840 \times 4 + 240$	772,976 B (388,288 B if packed)	773.0 kB
2 bit	$769,376 \times 1 + 840 \times 4 + 251$	772,987 B (195,955 B if packed)	773.0 kB

Table 7: On-disk size of each model

Each model's size is $N_w \times (\text{bytes per weight}) + N_b \times 4 \text{ B} + \text{the quantization-parameter file}$ ($N_w = 769,376$, $N_b = 840$). This gives 3,080,864 B for fp32 and 772,982 B for 8 bit, matching the file manager (3.1 MB and 773.0 kB, Figure 5). The 4-bit and 2-bit models also show 773.0 kB because we store each quantized value in one int8 byte. Packing the values would shrink them to about 388 kB and 196 kB.

6.3) Report the minimum amount of memory your system would need to successfully perform an end-to-end inference of each model (Note that this is a more nuanced question than it may initially appear to be, and heavily depends on your implementation). Don't go into extraordinary depth, but state your assumptions, thought process, and computation

Assumptions

1. Weights and biases stay in RAM
2. In each model, the input is fp32 ($64 \times 64 \times 3 \times 4 = 49,152$ bytes), dense2 output and softmax input/output is fp32 ($200 \times 4 = 800$ bytes each)
3. Quantized conv1 makes a temporary int8 copy of the image (12,288 bytes)

Thought Process

1. Our framework preallocates every layer's output buffer, which gives the 6.1 as-implemented memory (4,527 KB fp32 /1,185 KB quantized)
2. Inference runs layer by layer and each layer only reads the previous layers output so only the current input and output buffers are needed (ping pong buffers)
3. The minimum memory = parameters + max over layers of (input + output + temp) bytes
4. Peak layer
 - a. Fp32 conv2, gives $460,800 + 401,408 = 862,208$ bytes
 - b. Int8 conv2, gives $115,200 + 100,352 = 215,552$ bytes
 - c. Packed 4/2 bit conv1 becomes the peak because the fp32 image (49,152 bytes) does not shrink with bit width

- The mac loop uses a single int32 accumulator and the quant params take $8 \times 24 = 192$ bytes, so both are essentially negligible

Computation

Model	Parameters (bytes)	Peak Activations (bytes)	Total (kilobytes)
fp32	3,080,864	862,208 (conv2)	3,943
8 bit	772,736	215,552 (conv2)	988
4 bit	772,736	215,552 (conv2)	988
2 bit	772,736	215,552 (conv2)	988
4 bit packed	388,048	112,896 (conv1)	501
2 bit packed	195,704	81,024 (conv1)	277

Table 8: Minimum memory for end-to-end inference

6.4) Report how long inference takes. Is it the same for every image? Why or why not? Which is your fastest implementation? Explain the differences (or lack of differences) you see between each of your implementations

Model	Avg(ms)	Min(ms)	Max(ms)
fp32	142.9	142.1	150.4
8 bit	87.6	86.2	117.0
4 bit	87.6	86.2	107.3
2 bit	87.8	86.2	110.8

Table 9: Latency over 1,000 validation images

The min and average are nearly identical so in essence they are the same for 8, 4 and 2 bit. The difference of 1-2 ms is just a small amount of variability likely caused by outside factors to our model (caching). The network does the same number of MACs for every image, given there is no branching that depends on data. The fastest is pretty much a 3 way tie between 8-bit, 4-bit, and 2-bit.

The fp32 implementation is much more costly due to the computation and complexity of doing floating point arithmetic. This gives us a computation time 1.6x slower with no meaningful increase to overall accuracy.

The lack of difference is due to our implementation storing all values in the i8 format. We just clamp off the ends of the number we're computing with. We have a pseudo 4bit and 2 bit number. The CPU is doing the exact same amount of work by using a 8, 4, and 2 bit number and having an entirely different i4, i2 number would not accomplish much given the computation we do it in the same manner.

6.5) Take your 1000 of the inputs from the validation dataset and execute inference on each of your different quantization implementations and state & compare the top-1 and top-10 accuracy. State reasons for the observed differences if any.

Model	Top-1	Top-10
fp32	23.5%	60.5%
8 bit	23.8%	60.4%
4 bit	6.6%	24.1%
2 bit	0.3%	4.6%

Table 10: Top-1 and top-10 accuracy over 1,000 validation images

Measured Values are approximately equal for 8 bit and fp32. The difference of 0.3% is due to noise. Given 8 bit uses 1/4 the memory and runs 1.6x faster it is hard to argue for choosing fp32 over 8 bit quantized model. 4 bit and 2 bit drop off significantly because we must represent them in either 16 or 4 levels. This leads to compounded error since some values are set to 0. They fall out of consideration, giving us 8 bit as our best choice for a computational model.

6.6) You should note that for dequantizing the int32 accumulated value at each layer we divide by a real scalar constant. This is a costly operation. Is there a way to convert this to a int32 non float operation for the hardware that does not support the fp32 division?

Yes. In our implementation, each layer first dequantizes the int32 accumulator and then requantizes the result for the next layer:

$$y = \frac{acc}{S_i S_w}, \quad q_{out} = \text{round}(S_{out} \cdot y) + z_{out}$$

Both steps use constants that are known before inference, so they combine into a single multiplication:

$$q_{out} = \text{round}(M \cdot acc) + z_{out}, \quad M = \frac{S_{out}}{S_i S_w}$$

Following the paper linked in the manual (Jacob et al., 2018), the real multiplier M is written as a normalized fixed-point value and a power of two, and the fixed-point value is stored as an int32:

$$M = M_0 \cdot 2^{-n}, \quad 0.5 \leq M_0 < 1, \quad \hat{M}_0 = \text{round}(2^{31} \cdot M_0)$$

The division then becomes an integer multiply followed by a rounding right shift:

$$q_{out} = ((acc \times \hat{M}_0 + 2^{30+n}) \gg (31+n)) + z_{out}$$

This needs only an $\text{int32} \times \text{int32}$ multiply (with a 64-bit product), an add, and a shift. Both constants are computed offline for each layer. ReLU also stays in the integer domain, because the real value 0 maps to the zero-point:

$$q_{out} = \max(q_{out}, z_{out})$$

For example, for our conv1 layer:

$$M = \frac{75.75}{225.98 \times 419.31} = 7.994 \times 10^{-4} \Rightarrow n = 10, \quad \hat{M}_0 = 1,757,925,385$$

With $acc = 12,345$, the floating-point result is 9.87 and the integer-only version gives 10, the same value after rounding.